

Lecture 5: Unsupervised Learning I
PCA, t-SNE, Clustering, Autoencoders
Blood Cells, Fraud and Tumour Genes

Deep Learning for Scientific Discovery

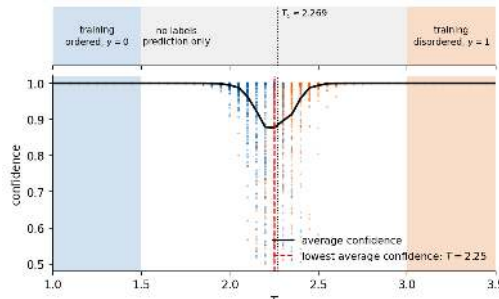
Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków



Recap: Finding T_c with Labels

- a CNN trained on spin configurations labelled "ordered" ($T < 1.5$) and "disordered" ($T > 3$)
- its average confidence is lowest at $T = 2.25$ ($L = 40$); exact value $T_c \approx 2.269$
- but the labels came from us: we knew that there are two phases, and where each of them lies



Today: what can we learn from data without labels?

1

Learning without Labels



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Unsupervised Learning

Supervised learning needs labels, and labels are expensive (often an expert labels every sample), they contain what we already know (the ten digits, the two phases of a magnet), and in discovery the categories are often unknown.

Supervised

data: pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$

goal: a function $f(\mathbf{x}) \approx y$, i.e. learn $p(y \mid \mathbf{x})$

the question is fixed in advance: which cell type? which phase?

Unsupervised

data: samples only, $\{\mathbf{x}_i\}_{i=1}^n$

goal: learn about $p(\mathbf{x})$, the distribution the data come from

no question fixed in advance: we want to **understand the data**

Questions about Data

question about the data	method
which few variables describe a sample?	dimensionality reduction
how many kinds of samples are there?	clustering
what is typical, and what is unusual?	anomaly detection
how are the samples distributed?	density estimation,
can we draw new ones?	generative models

The method proposes a structure; deciding what it means is ours.

High-Dimensional Data

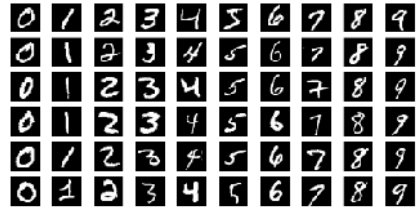
every sample is a vector $\mathbf{x} \in \mathbb{R}^D$: an MNIST digit has $D = 28 \cdot 28 = 784$ pixels, an Ising configuration $D = L^2$ spins

We cannot look at 784 dimensions. But the data do not fill $\mathbb{R}^{784}$: pick 784 random pixel values and you get noise, never a digit. Real images lie in a thin, low-dimensional part of the space.

So the analysis starts with **dimensionality reduction**: project the data on a smaller space,

$$\mathbf{x} \in \mathbb{R}^D \;\mapsto\; \mathbf{z} \in \mathbb{R}^d, \qquad d \ll D,$$

keeping their structure; look at the points $\mathbf{z}_i$: do they form groups?



MNIST: 70 000 handwritten digits 0–9 (60 000 training, 10 000 test), 28×28 grey values

Y. LeCun et al., Proc. IEEE **86**, 2278 (1998), doi:10.1109/5.726791

2

Principal Component Analysis

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Principal Component Analysis: the Covariance Matrix

Data $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^D$; x_{ij} is feature j (e.g. pixel j) of sample i. The mean $\bar{\mathbf{x}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{x}_i$ (for MNIST: the average image), and the **covariance matrix** of the features, a $D \times D$ matrix:

$$C_{jl} = \frac{1}{n}\sum_{i=1}^{n}(x_{ij} - \bar{x}_j)(x_{il} - \bar{x}_l), \qquad j, l = 1, \ldots, D$$

- C_{jj}: the variance of feature j
- C_{jl}: how features j and l vary together over the data (> 0: rise together; < 0: one rises, the other falls); with each feature divided by its standard deviation, C becomes the *correlation* matrix

K. Pearson, *On lines and planes of closest fit to systems of points in space*, Phil. Mag. **2**, 559 (1901), doi:10.1080/14786440109462720; H. Hotelling, *Analysis of a complex of statistical variables into principal components*, J. Educ. Psychol. **24**, 417 (1933), doi:10.1037/h0071325

Principal Component Analysis: a Linear Projection

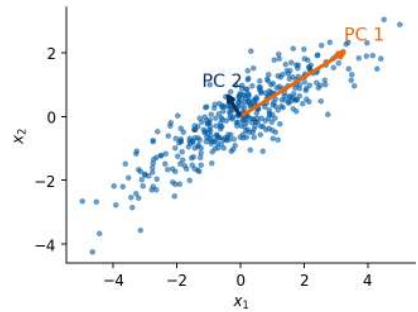
- diagonalise C:

$$C\,\boldsymbol{\lambda}_l = \sigma_l^2\,\boldsymbol{\lambda}_l, \qquad \sigma_1^2 \geq \sigma_2^2 \geq \cdots \geq \sigma_D^2 \geq 0$$

  $\boldsymbol{\lambda}_l \in \mathbb{R}^D$: the l-th eigenvector, a unit vector ($\boldsymbol{\lambda}_l^\top \boldsymbol{\lambda}_l = 1$)

- the variance of the data along a unit vector $\mathbf{u}$ is $\mathbf{u}^\top C\,\mathbf{u}$; it is largest for $\mathbf{u} = \boldsymbol{\lambda}_1$, with value σ_1^2

- keep the L eigenvectors with the largest variances σ_l^2: the **principal components**



400 correlated points: PC 1 carries 94.5% of the variance

PCA: New Coordinates of Every Sample

Centre the data, $\mathbf{x}_i \to \mathbf{x}_i - \bar{\mathbf{x}}$. The l-th new coordinate of sample i is its projection on the l-th principal direction:

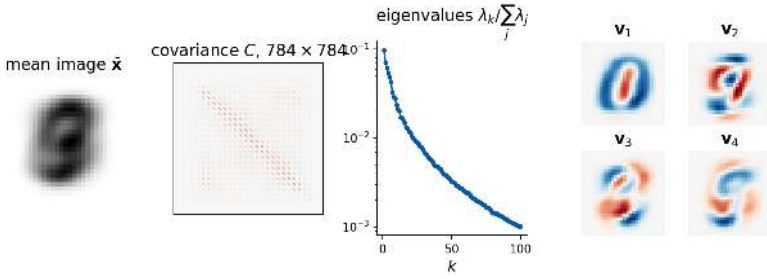
$$w_i^{(l)} = \boldsymbol{\lambda}_l^\top \mathbf{x}_i = \sum_{j=1}^{D} \lambda_{lj}\, x_{ij}, \qquad l = 1, \ldots, L$$

(λ_{lj}: component j of $\boldsymbol{\lambda}_l$). The vector of the L projections is the **new representation** of $\mathbf{x}_i$:

$$\mathbf{w}_i = \left[w_i^{(1)}, \ldots, w_i^{(L)}\right]^\top = \Lambda^\top \mathbf{x}_i, \qquad \Lambda = \left(\boldsymbol{\lambda}_1, \ldots, \boldsymbol{\lambda}_L\right) \in \mathbb{R}^{D \times L}$$

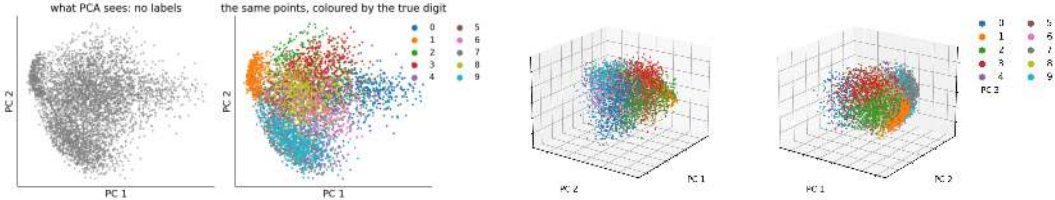
- $\mathbf{x}_i \in \mathbb{R}^D$ (784 pixels) $\to$ $\mathbf{w}_i \in \mathbb{R}^L$ ($L = 2$: a point of the plane we can plot)
- one fixed linear map for all samples: a *new* image is mapped with the same Λ
- variance of $w^{(l)}$ over the data: σ_l^2, largest for $l = 1$

PCA on MNIST: Covariance and Eigenvectors



- 60 000 training images of 784 pixels; C is 784×784: how two pixels vary together over the images
- the eigenvalues fall fast: 2 components keep 17% of the variance, 10 keep 49%, 87 keep 90%
- the eigenvectors are themselves images: patterns of pixels that vary together

PCA on MNIST



- every point is the new representation $\mathbf{w}_i = [w_i^{(1)}, w_i^{(2)}]$ of one of 5000 test digits (left): the first two components keep 17% of the variance; three components (right, two viewing angles) keep 23%; 50 components keep 82%
- is there clustering? some digits form separate groups (0, 1), the third direction separates a few more, but most overlap: the digits do not lie in a flat, low-dimensional subspace, and a linear projection is too crude

3
t-SNE

JAGIELLONIAN UNIVERSITY
IN KRAKOW

t-SNE: Similarities in the Data

Neighbours in $\mathbb{R}^D$ stay neighbours in the 2D map; the map positions are the parameters, found by gradient descent (no network). Instances $\mathbf{x}_i \in \mathbb{R}^D$, $i = 1, \ldots, n$ (an MNIST image: $D = 784$), squared distance $\|\mathbf{x}_i - \mathbf{x}_j\|^2 = \sum_{k=1}^{D} (x_{ik} - x_{jk})^2$. For instance i, a Gaussian of width σ_i turns distances into neighbour probabilities over the $n - 1$ other instances:

$$p_{j|i} = \frac{e^{-\|\mathbf{x}_i - \mathbf{x}_j\|^2 / 2\sigma_i^2}}{\sum_{l \neq i} e^{-\|\mathbf{x}_i - \mathbf{x}_l\|^2 / 2\sigma_i^2}}, \qquad p_{i|i} = 0, \qquad p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$$

- $p_{j|i}$: the probability that $\mathbf{x}_i$ picks $\mathbf{x}_j$ as its neighbour; σ_i is chosen so that the **perplexity** 2^{H_i}, $H_i = -\sum_j p_{j|i} \log_2 p_{j|i}$, equals a set value (here 30): the effective number of neighbours (k equally likely neighbours give $2^{H_i} = k$)
- $P = \{p_{ij}\}$, $\sum_{i \neq j} p_{ij} = 1$: a probability distribution over pairs, how likely i and j are neighbours *in the original space*

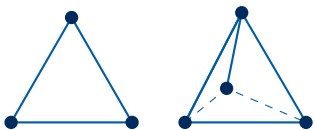
t-SNE: Similarities in the Map

$\mathbf{y}_i \in \mathbb{R}^d$: the point that represents instance i in a map of dimension d we choose ($d = 2$ to plot it); the $\mathbf{y}_i$ are the unknowns.

$$q_{ij} = \frac{(1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}}{\sum_{l \neq m}(1 + \|\mathbf{y}_l - \mathbf{y}_m\|^2)^{-1}}, \qquad \|\mathbf{y}_i - \mathbf{y}_j\|^2 = \sum_{a=1}^{d}(y_{ia} - y_{ja})^2$$

- the numerator is a Student-t kernel of the map distance; in the original space the kernel was a Gaussian
- the denominator sums over all pairs, so $\sum_{i \neq j} q_{ij} = 1$: $Q = \{q_{ij}\}$ is the same kind of distribution over pairs, *in the map*
- t-SNE moves the $\mathbf{y}_i$ until Q matches P: pairs that are neighbours in $\mathbb{R}^D$ (large p_{ij}) must get large q_{ij}, i.e. be close in the map

The Crowding Problem



plane: at most 3 3D: 4

points all at the *same* distance from each other: $D+1$ in $\mathbb{R}^D$, so 785 in the MNIST pixel space, but only 3 in a 2D map

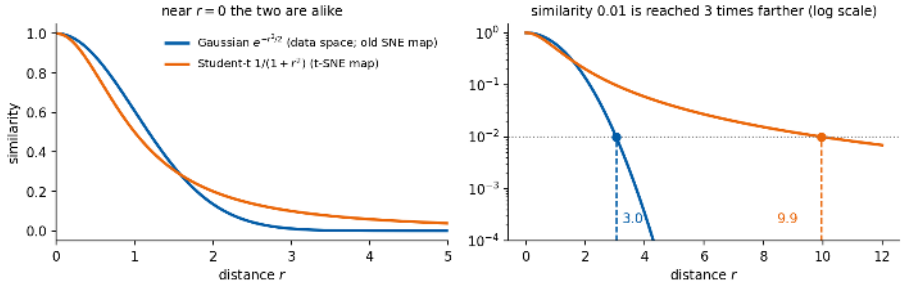
Too little room. Groups roughly equally far apart in $\mathbb{R}^{784}$ (the ten digits) cannot all be equally far apart in the plane: some pairs at a moderate distance must be placed much farther away.

With a Gaussian in the map (the older SNE): a pair placed far away gets $q_{ij} \approx 0$, although its p_{ij} is small but not zero. The cost pulls such pairs back, and all groups are squeezed into the centre of the map: the *crowding problem*.

L. van der Maaten, G. Hinton, *Visualizing Data using t-SNE*, J. Mach. Learn. Res. **9**, 2579 (2008),

jmlr.org/papers/v9/vandermaaten08a

The Heavy Tail of the Student-t



The Student-t decays like $1/r^2$, the Gaussian like $e^{-r^2/2}$. A small similarity such as 0.01 is reached at $r = 3.0$ with the Gaussian but at $r = 9.9$ with the Student-t. So a moderately distant pair (small p_{ij}) can be placed far apart in the map and still get a matching q_{ij}: there is room to push groups apart, and **gaps open between them**.

L. van der Maaten, G. Hinton, *Visualizing Data using t-SNE*, J. Mach. Learn. Res. **9**, 2579 (2008), jmlr.org/papers/v9/vandermaaten08a

t-SNE: the Loss

Two distributions over pairs of instances: P from the original space, Q from the map. Move the map points $\mathbf{y}_i$ by gradient descent until Q matches P, i.e. minimise their Kullback–Leibler divergence:

$$\mathcal{L}(\mathbf{y}_1, \ldots, \mathbf{y}_n) = \mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

- neighbours placed far apart (p_{ij} large, q_{ij} small): large cost
- distant instances placed close (p_{ij} small): small cost
- so t-SNE keeps the **local** structure; distances between groups in the map carry little meaning

The result is a set of 2D points, one per instance, that we can plot and inspect.

L. van der Maaten, G. Hinton, *Visualizing Data using t-SNE*, J. Mach. Learn. Res. **9**, 2579 (2008), jmlr.org/papers/v9/vandermaaten08a

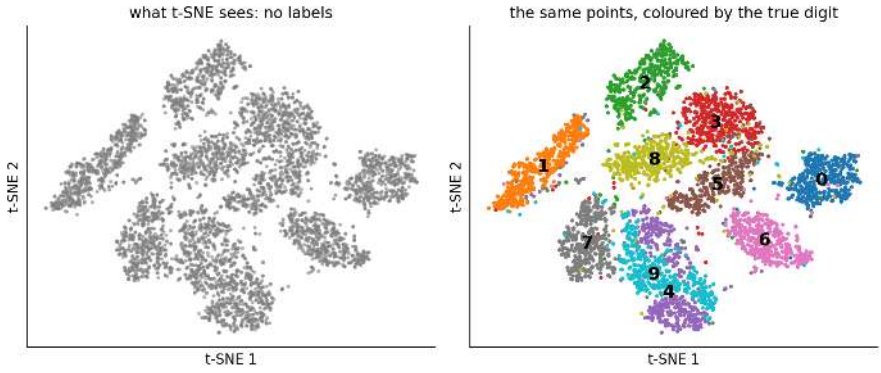
t-SNE: New Coordinates of Every Sample

Every sample $\mathbf{x}_i \in \mathbb{R}^D$ gets a new representation, its map point

$$\mathbf{y}_i = \left[y_i^{(1)}, y_i^{(2)}\right]^\top \in \mathbb{R}^2, \qquad i = 1, \ldots, n$$

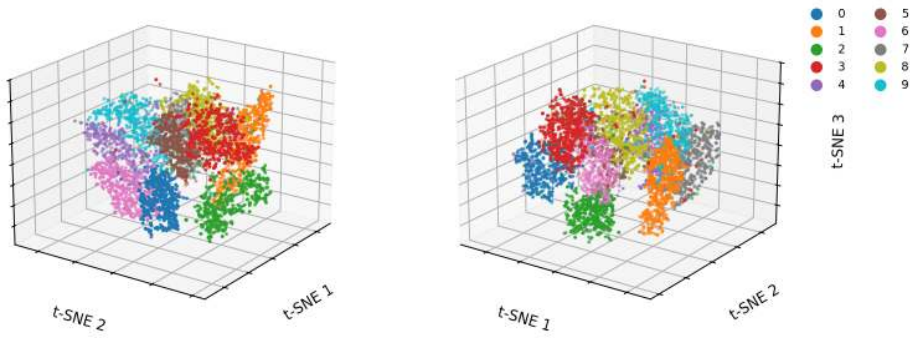
- PCA: one formula for all samples, $\mathbf{w}_i = \Lambda^\top \mathbf{x}_i$
- t-SNE: no formula $\mathbf{y}_i = f(\mathbf{x}_i)$; the $2n$ numbers $y_i^{(a)}$ are themselves the parameters, found together by minimising $\mathrm{KL}(P \,\|\, Q)$
- a coordinate $y_i^{(1)}$ has no meaning on its own: only the neighbourhoods of the points do; rotating or mirroring the map gives the same loss
- a new sample has no map point until t-SNE is run again with it

t-SNE on MNIST



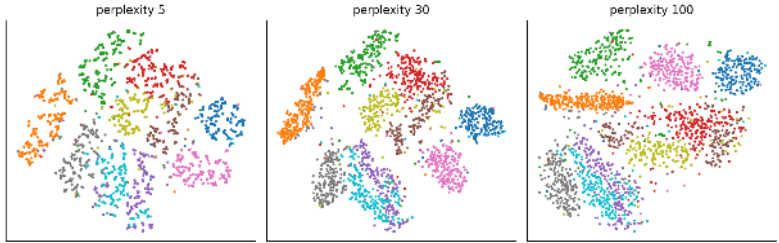
- 5000 test digits, $d = 2$; the labels only colour the right panel
- ten groups appear without any label

t-SNE on MNIST in 3D



The same loss with $\mathbf{y}_i \in \mathbb{R}^3$: the 5000 digits seen from two angles.

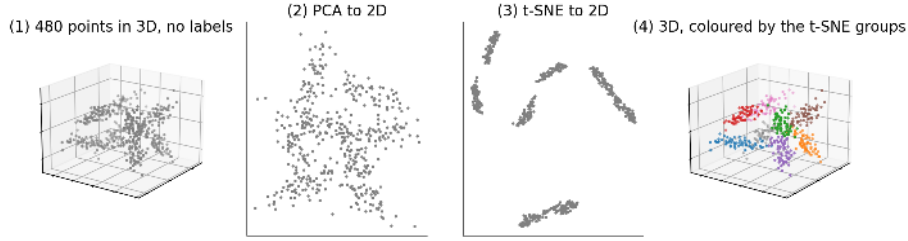
The t-SNE Map



2000 digits at three perplexities: small perplexity keeps only very close neighbours, large perplexity smooths the map

- **meaningless:** the position of a cluster (a rotated, mirrored or shifted map has the same loss); the separation between clusters (distant pairs have $p_{ij} \approx 0$); cluster sizes (σ_i adapts to the density)
- **reliable:** which instances are close neighbours

t-SNE on Data in 3D



- 480 points in $\mathbb{R}^3$, *without* labels (1): eight elongated groups passing through each other; neither the 3D view nor PCA (2) shows how many there are
- t-SNE keeps neighbours together: its 2D map (3) shows eight separate groups; coloured back in 3D (4), they are the groups that generated the data

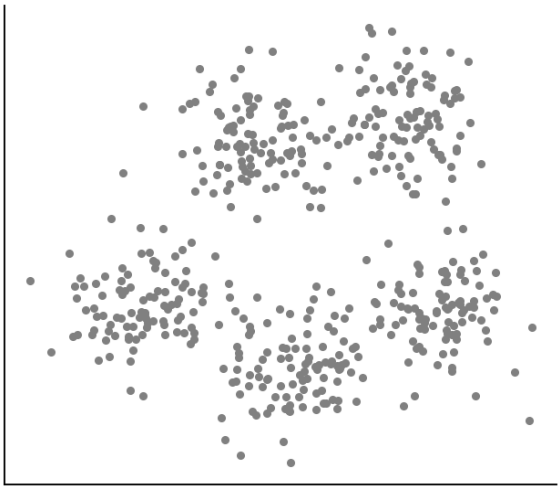
4

Clustering



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Points without Labels



500 points in 2D, no labels.

We see some groups. How many are there, and where is the boundary between two of them?

k-means Clustering

Goal: split the n instances $\mathbf{x}_1, \dots, \mathbf{x}_n$ into k clusters $S_1, \dots, S_k$ (sets of instance indices) with centres $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k$, minimising the total squared distance of the instances to their centres:

$$\mathcal{L} = \sum_{c=1}^{k} \sum_{i \in S_c} \|\mathbf{x}_i - \boldsymbol{\mu}_c\|^2$$

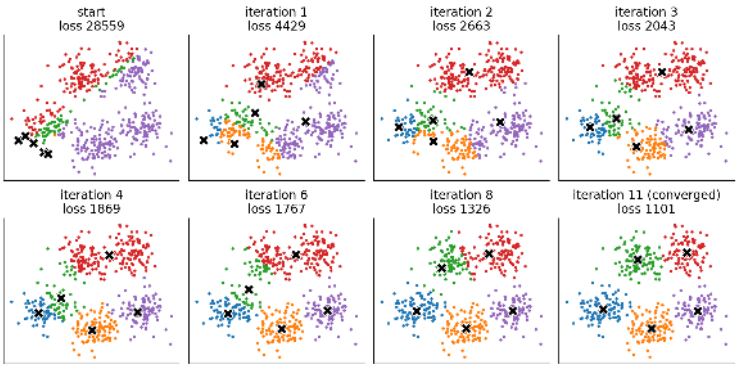
Algorithm: start from k centres; repeat until no instance changes cluster:

- *assign* each instance to its nearest centre:
  $S_c = \{\, i : \|\mathbf{x}_i - \boldsymbol{\mu}_c\| \le \|\mathbf{x}_i - \boldsymbol{\mu}_{c'}\| \text{ for all } c' \,\}$
- *update* each centre to the mean of its instances: $\boldsymbol{\mu}_c = \frac{1}{|S_c|} \sum_{i \in S_c} \mathbf{x}_i$

No move raises $\mathcal{L}$; keep the best of several random starts. k is chosen by us.

S. P. Lloyd, *Least squares quantization in PCM*, IEEE Trans. Inf. Theory **28**, 129 (1982), doi:10.1109/TIT.1982.1056489

k-means Iterations

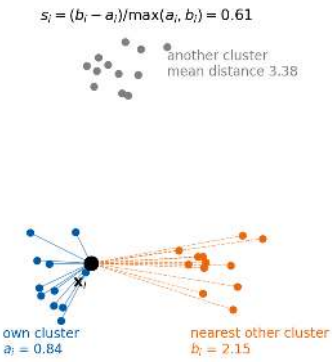


the same 500 points, $k = 5$

start: five centres (crosses) placed badly, at the five points nearest the lower-left corner; colours: the assignment

the loss falls from 28 559 to 1 101; after 11 iterations nothing changes, and the clusters match the five groups that generated the points (ARI 0.93)

The Silhouette Score



In a good clustering each instance is close to its own cluster and far from the others. For $\mathbf{x}_i$ in cluster S_c:

- a_i: mean distance to the other instances of S_c
- b_i: mean distance to the instances of the nearest other cluster

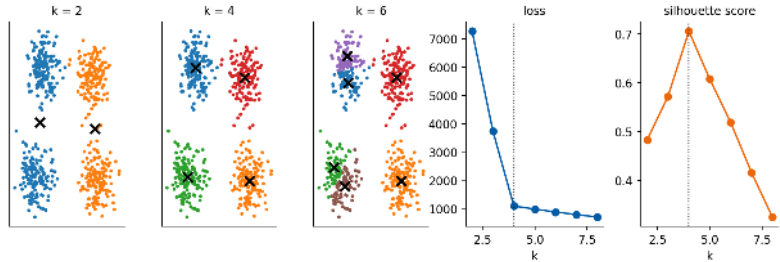
$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)} \in [-1, 1]$$

$s_i \approx 1$: well inside its cluster; $s_i \approx 0$: on a border; $s_i < 0$: probably in the wrong cluster.

Mean silhouette $\bar{s}(k) = \frac{1}{n}\sum_i s_i$ for the clustering with k clusters: choose the k with the largest $\bar{s}(k)$.

P. J. Rousseeuw, *Silhouettes: A graphical aid to the interpretation and validation of cluster analysis*, J. Comput. Appl. Math. **20**, 53 (1987), doi:10.1016/0377-0427(87)90125-7

Choosing the Number of Clusters



Four groups, so the right answer is known. For $k = 2, \ldots, 8$: run k-means (10 random starts), record the loss $\mathcal{L}(k)$ and the mean silhouette $\bar{s}(k)$.

- **elbow:** $\mathcal{L}(k)$ always decreases with k; fast up to the right k, slowly after it: the bend is at $k = 4$
- **silhouette:** $\bar{s}(k)$ is highest at $k = 4$; too small k merges groups, too large k splits them

Rand Index

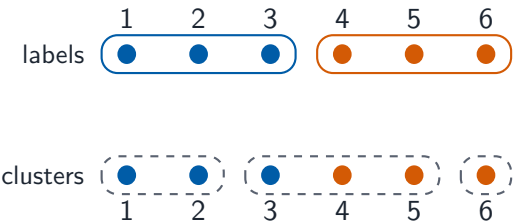
Compare a clustering with known classes *pair by pair*; the names of the groups do not matter.

A pair of instances (i, j) **agrees** if both partitions put it together, or both put it apart. Out of all $n(n-1)/2$ pairs:

- a: pairs together in both
- b: pairs apart in both

$$\mathrm{RI} = \frac{a + b}{n(n-1)/2} \in [0, 1]$$

$\mathrm{RI} = 1$: identical partitions; every disagreeing pair lowers it.



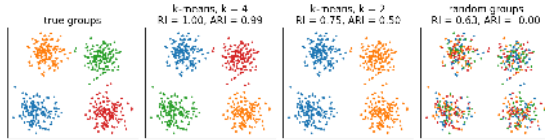
together in both: 1–2, 4–5: $a = 2$

apart in both: the other 7 pairs: $b = 7$

together in one only (disagree):
1–3, 2–3, 3–4, 3–5, 4–6, 5–6

$$\mathrm{RI} = (2 + 7)/15 = 0.60$$

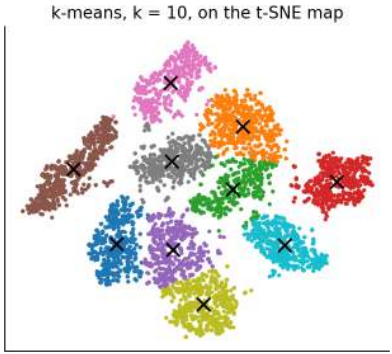
Adjusted Rand Index



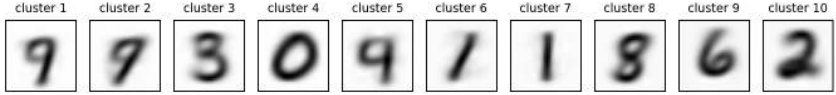
- random groups already reach $\mathrm{RI} = 0.63$: most pairs are apart in both partitions
- the adjusted Rand index subtracts what chance gives, $\mathrm{ARI} = \dfrac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{1 - \mathbb{E}[\mathrm{RI}]}$; $\mathbb{E}[\mathrm{RI}]$: the mean RI when the instances are reassigned at random, keeping the group sizes
- **ARI = 1**: the same partition; **ARI $\approx$ 0**: no better than chance (it can be negative); the six points of the sketch: 0.12
- the eight groups in 3D, k-means ($k = 8$): ARI 0.46 on the PCA projection, 0.96 on the t-SNE map

L. Hubert and P. Arabie, *Comparing partitions*, J. Classification **2**, 193 (1985), doi:10.1007/BF01908075

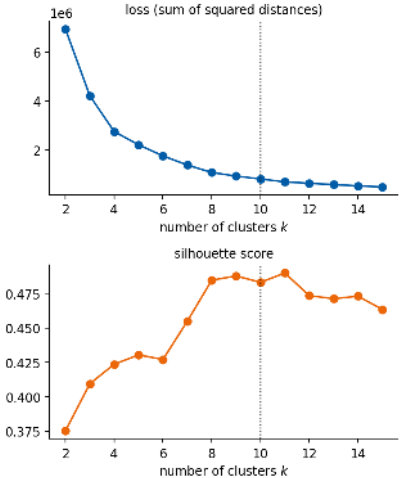
k-means on MNIST



- $k = 10$, the number of digits; 5000 test digits
- agreement with the true digits, ARI: **0.38** on 50 PCA components, **0.76** on the t-SNE map
- below: the ten centres $\boldsymbol{\mu}_c$ found on 50 PCA components, shown as images

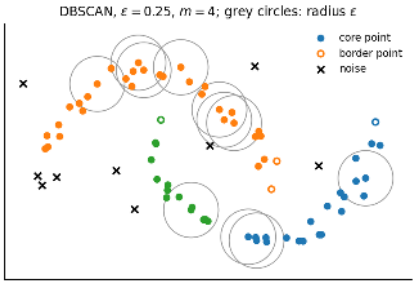


Choosing k on MNIST



- the loss $\mathcal{L}(k)$ and the mean silhouette $\bar{s}(k)$ on the t-SNE map of the 5000 digits
- dotted line: $k = 10$, the number of digits, drawn only for comparison
- no sharp elbow; the silhouette is highest at $k = 11$ (one digit split in two)
- on real data the guides suggest a range of k; the final choice uses what we know about the data

DBSCAN



Density-Based Spatial Clustering of Applications with Noise. Parameters: a radius ε, a minimum number of points m; the ε-neighbourhood of instance i (itself included):

$$N_\varepsilon(i) = \{\, j : \|\mathbf{x}_j - \mathbf{x}_i\| \leq \varepsilon \,\}$$

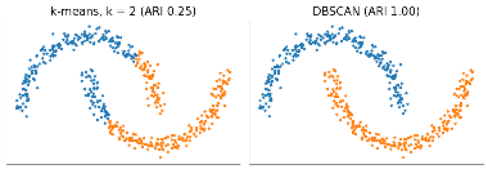
- **core point:** $|N_\varepsilon(i)| \geq m$ (grey circles)
- **border point:** not core, but $i \in N_\varepsilon(j)$ for some core point j
- **noise:** all other points (crosses)

A cluster: core points linked by chains of core neighbours, plus their border points.

M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*, KDD 1996, AAAI proceedings

DBSCAN: the Algorithm

1. compute $N_\varepsilon(i)$ for every instance and mark the core points
2. take a core point not yet in a cluster and start a new cluster with it
3. add every point of $N_\varepsilon(j)$ for every core point j of the cluster; repeat until no point is added: the cluster grows through the dense region and stops at its border points
4. repeat 2 and 3 until every core point is in a cluster; points never reached are noise



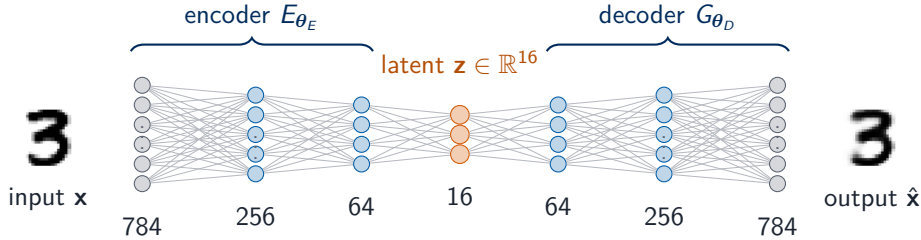
k-means cuts the space into convex cells, so it finds round clusters: on two half-moons k-means ($k = 2$) gives ARI 0.25, DBSCAN 1.00. DBSCAN needs no number of clusters, and clusters can have any shape.

5
Autoencoders

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Autoencoder

If a network can rebuild an image from only d numbers, those numbers contain what is essential about it (PCA needs 87 components for 90% of the MNIST variance).



- **encoder**, a network $\mathbb{R}^D \to \mathbb{R}^d$ with parameters $\boldsymbol{\theta}_E$: $\mathbf{z} = E_{\boldsymbol{\theta}_E}(\mathbf{x})$; **decoder**, $\mathbb{R}^d \to \mathbb{R}^D$ with $\boldsymbol{\theta}_D$: $\hat{\mathbf{x}} = G_{\boldsymbol{\theta}_D}(\mathbf{z})$
- the bottleneck $d \ll D$ forces $\mathbf{z}$ to keep the features the decoder needs: a learned **latent-space representation**

G. E. Hinton, R. R. Salakhutdinov, Science **313**, 504 (2006), doi:10.1126/science.1127647

The Autoencoder Loss



Each input $\mathbf{x}_i$ is compared with its reconstruction $\hat{\mathbf{x}}_i = G_{\theta_D}(E_{\theta_E}(\mathbf{x}_i))$, pixel by pixel:

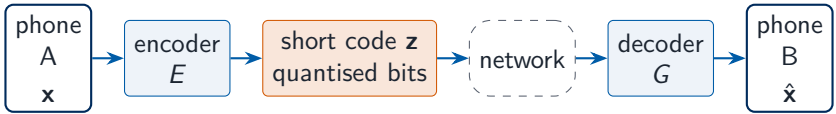
$$\mathcal{L}(\boldsymbol{\theta}_E, \boldsymbol{\theta}_D) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{D}\sum_{j=1}^{D}\left(x_{ij} - \hat{x}_{ij}\right)^2$$

i: image, j: pixel (right panel: one digit). No labels: the target is the input itself.

- trained by gradient descent; MNIST: dense layers $784 \to 256 \to 64 \to d$ and back
- a linear autoencoder with squared loss finds the PCA subspace (Baldi & Hornik 1989); non-linear layers can follow curved structure

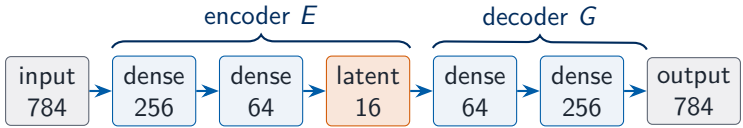
P. Baldi, K. Hornik, *Neural networks and principal component analysis: Learning from examples without local minima*, Neural Networks **2**, 53 (1989), doi:10.1016/0893-6080(89)90014-2

Compression on a Phone



- **photos today:** JPEG and HEIC follow the same pattern, encode → short code → decode, with a fixed transform designed by engineers (JPEG: discrete cosine transform and quantisation), not a trained network
- **learned image codecs:** an autoencoder whose code is quantised and entropy-coded, its length set by a learned prior (research: Ballé et al., ICLR 2018); a standard has been published, JPEG AI, ISO/IEC 6048-1:2025
- **speech:** Google released the Lyra codec, which uses the SoundStream neural codec, runs at 3.2–9.2 kbit/s and is designed to run in real time on phones; Google reports SoundStream at 3 kbit/s outperforming Opus at 12 kbit/s

Dense Autoencoder: Layers

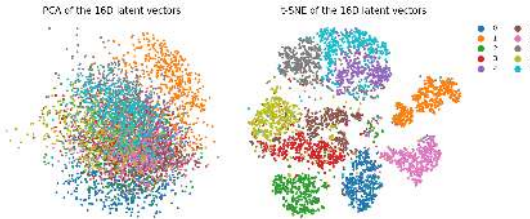


encoder	output	parameters
Linear	256	200 960
Linear	64	16 448
Linear	16	1 040

decoder	output	parameters
Linear	64	1 088
Linear	256	16 640
Linear	784	201 488

total 437 664 parameters, all trainable, none frozen; ReLU after every hidden layer, sigmoid at the output (no parameters, not listed)

Compression and the Latent Space



5000 test digits, $d = 16$ latent numbers each, drawn in 2D; colours: true digits, used only for plotting

- **compression:** 784 pixels become 16 numbers $\mathbf{z} = E_{\theta_E}(\mathbf{x})$; the encoder is a continuous function, so similar images get nearby $\mathbf{z}$, and the bottleneck keeps only what the decoder needs
- **PCA of z** overlaps the digits; **t-SNE of z** keeps the neighbours in 16D and shows the groups

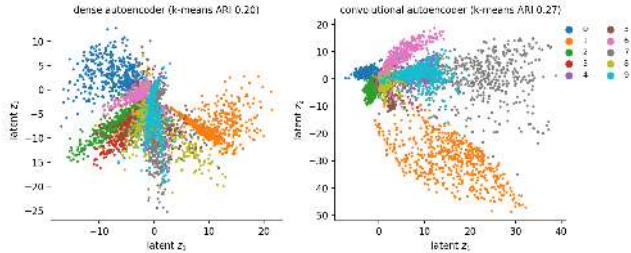
k-means in the Latent Space

k-means ($k = 10$) against the true digits, adjusted Rand index:

	directly	on the t-SNE map
pixels (50 PCA components)	0.38	0.76
16 latent numbers of the autoencoder	0.28	0.67

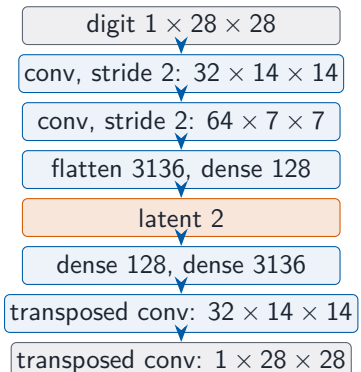
- the digits are kept in the latent space, but not as the round groups that k-means looks for
- t-SNE only *draws* the latent space; the latent space itself takes *new* images (encoder) and gives back images (decoder)

Dense and Convolutional Autoencoders



- $d = 2$ latent numbers, so the latent space is drawn directly; colours: true digits
- **dense** (left): a flat list of 784 pixels; **convolutional** (right): two convolutions with stride 2 ($28 \to 14 \to 7$ pixels, 32 and 64 channels)
- k-means ($k = 10$), ARI: 0.20 (dense), 0.27 (convolutional); in this run (one seed) the convolutional encoder, which compares neighbouring pixels, separates the digits better

Convolutional Autoencoder: Layers



layer	output	parameters
Conv2d	$32 \times 14 \times 14$	320
Conv2d	$64 \times 7 \times 7$	18 496
Linear	128	401 536
Linear	2	258
Linear	128	384
Linear	3136	404 544
ConvTranspose2d	$32 \times 14 \times 14$	18 464
ConvTranspose2d	$1 \times 28 \times 28$	289
total (all trainable)		844 291

upper block: encoder, lower block: decoder; ReLU, flatten, unflatten and the output sigmoid have no parameters and are not listed

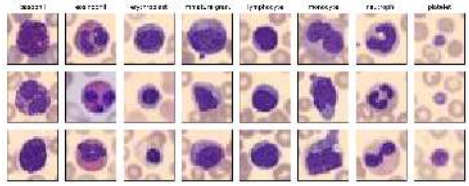
6

Blood Cells without Labels



JAGIELLONIAN UNIVERSITY
IN KRAKOW

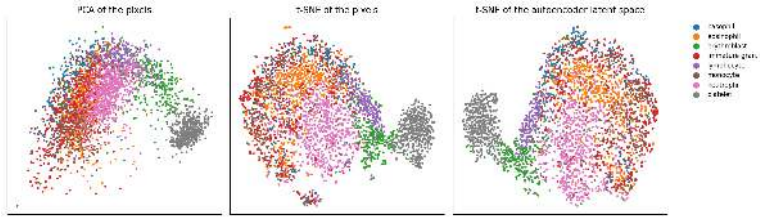
BloodMNIST



- microscope images of single *normal* blood cells, eight types assigned by clinical pathologists; 28×28 pixels, 3 colours: $D = 2352$; 11 959 training and 3421 test images
- white blood cells: basophil, eosinophil, neutrophil, lymphocyte, monocyte, immature granulocyte (not yet mature); erythroblast: an immature red blood cell; platelet: a small cell fragment

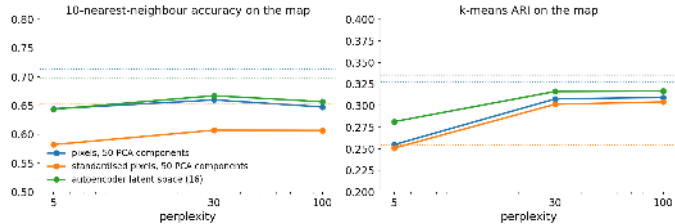
A. Acevedo et al., *A dataset of microscopic peripheral blood cell images for development of automatic recognition systems*, Data in Brief **30**, 105474 (2020), doi:10.1016/j.dib.2020.105474; J. Yang et al., *MedMNIST v2*, Sci. Data **10**, 41 (2023), doi:10.1038/s41597-022-01721-8

Blood Cells without Labels



- the 3421 BloodMNIST test images; left to right: PCA of the pixels, t-SNE of the pixels, t-SNE of the 16 latent numbers of a convolutional autoencoder trained on the images alone (cell types only colour the points)
- k-means ($k = 8$) against the cell types, ARI: 0.33 (50 PCA components), 0.34 (latent space)
- platelets and erythroblasts form their own groups; the white-blood-cell types overlap, in the pixels and in the latent space

t-SNE Settings for Blood Cells



- perplexity 5, 30, 100 on three inputs; score: vote of the 10 nearest neighbours (chance 0.20) and k-means ARI; dotted: the input before t-SNE
- no setting separates the eight types: vote 0.58–0.67 on the maps, 0.65–0.71 on the inputs; ARI at most 0.32
- t-SNE keeps neighbours, and the neighbours are already mixed in the input; possibly pixels mostly carry colour, size and brightness, while the white-cell types differ in finer detail

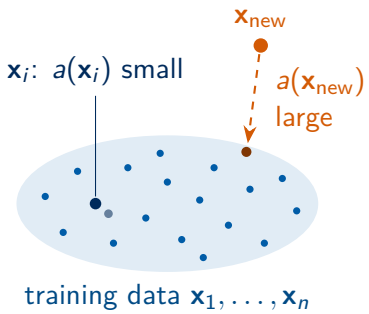
7

Anomaly Detection



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Anomaly Detection with Reconstruction Error



light dots: reconstructions $\hat{\mathbf{x}}$

Train the autoencoder on the data $\mathbf{x}_1, \ldots, \mathbf{x}_n$:

$$\mathcal{L} = \frac{1}{nD} \sum_{i=1}^{n} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2, \qquad \hat{\mathbf{x}}_i = G_{\theta_D}(E_{\theta_E}(\mathbf{x}_i))$$

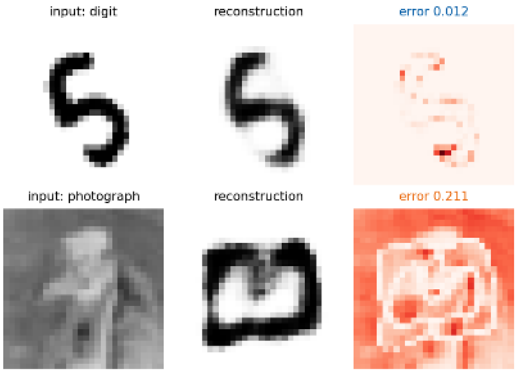
so for typical data $\hat{\mathbf{x}}_i \approx \mathbf{x}_i$. The decoder produces only what it learned: a new $\mathbf{x}$ unlike the training data is mapped back towards them, far from itself.

Anomaly score of any $\mathbf{x}$, and the decision:

$$a(\mathbf{x}) = \frac{1}{D} \|\mathbf{x} - G_{\theta_D}(E_{\theta_E}(\mathbf{x}))\|^2,$$

anomaly if $a(\mathbf{x}) > \tau$; τ: e.g. exceeded by 5% of normal samples (5% false alarms).

A Digit and a Photograph

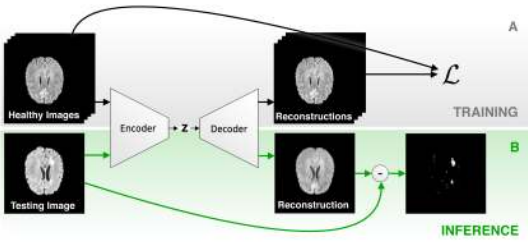


shown: the median case of each kind

- autoencoder trained on all MNIST digits
- photographs: CIFAR-10 test images (Lecture 4), grey, 28×28
- a typical digit: rebuilt well, small $a(\mathbf{x})$
- a photograph: rebuilt as a blurred, digit-like image, large $a(\mathbf{x})$: an **anomaly**

A. Krizhevsky, *Learning multiple layers of features from tiny images*, University of Toronto (2009), cs.toronto.edu/~kriz/cifar.html

Anomaly Detection in Practice

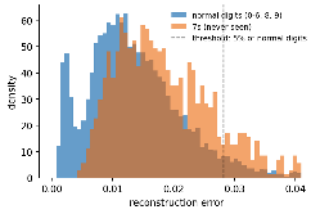


- **medical imaging:** an autoencoder trained on brain MRI of healthy people (A) cannot rebuild a *lesion* (a region of abnormal tissue, e.g. a tumour); scan minus reconstruction shows where it is (B)
- **fraud**: a card transaction that does not fit the usual patterns
- **science**: a faulty measurement

C. Baur et al., *Autoencoders for unsupervised anomaly segmentation in brain MR images: A comparative study*, Med. Image Anal. **69**, 101952 (2021), doi:10.1016/j.media.2020.101952; V. Chandola, A. Banerjee and V. Kumar, *Anomaly detection: A survey*, ACM Comput. Surv. **41**, 15 (2009), doi:10.1145/1541880.1541882

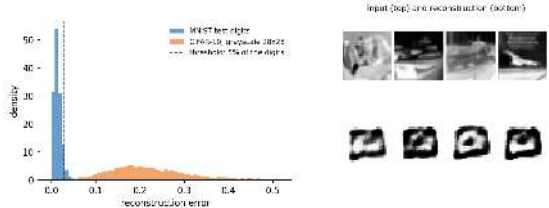
Two Anomalies

subtle: the digit 7



- trained without the 7s; blue: trained digits, orange: 7s
- 5% false alarms flag only 13% of the 7s (AUC 0.68), possibly because a 7 shares strokes with 1 and 9

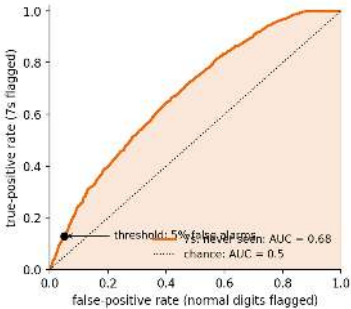
clear: photographs



- trained on all digits; input: CIFAR-10 photographs, 28×28 gray
- the same rule flags 99.8% of them (AUC 0.999)

ROC Curve and AUC

Purpose: compare anomaly scores without choosing a threshold first.



n_A anomalies and n_N normal samples; for a threshold τ, $k_A(\tau)$ anomalies and $k_N(\tau)$ normal samples have $a > \tau$:

$$\mathrm{TPR}(\tau) = \frac{k_A(\tau)}{n_A}, \qquad \mathrm{FPR}(\tau) = \frac{k_N(\tau)}{n_N}$$

(anomalies found, false alarms). Lowering τ traces the **ROC curve** from $(0, 0)$ to $(1, 1)$; chance: the diagonal.

$$\mathrm{AUC} = P\big(a(\mathbf{x}_{\text{anomaly}}) > a(\mathbf{x}_{\text{normal}})\big)$$

area under the curve (random anomaly vs random normal sample): 0.5 for chance, 1 for perfect separation

Credit-Card Fraud Data

	V1	V2	⋯	V28	ln(1+A)	Class
	-1.36	-0.07	⋯	-0.02	5.01	0
	1.19	0.27	⋯	0.01	1.31	0
$\mathbf{x}_i$	-1.36	-1.34	⋯	-0.06	5.94	0
	-0.97	-0.19	⋯	0.06	4.82	0
	-1.16	0.88	⋯	0.22	4.26	0
			⋮			

$D = 29$ features: $\mathbf{x}_i = (x_{i1}, \ldots, x_{i,29})$

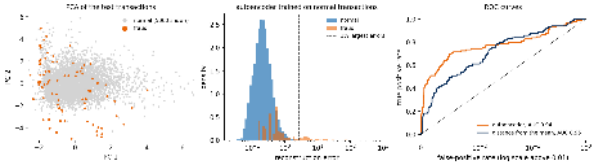
only to evaluate

One row per transaction (first five shown): $\mathbf{x}_i$ is a vector of 29 numbers.

- 284 807 transactions of European cardholders, two days in September 2013; 492 frauds (0.17%)
- `V1`–`V28`: principal components of original features not released (confidentiality), so no readable meaning; A: amount
- `Class` (1 = fraud): only to evaluate

A. Dal Pozzolo, O. Caelen, R. A. Johnson, G. Bontempi, *Calibrating Probability with Undersampling for Unbalanced Classification*, IEEE SSCI 2015, 159, doi:10.1109/SSCI.2015.33; data: OpenML 1597

Fraud Detection by Reconstruction Error



- **PCA:** two components keep 10% of the variance; many frauds lie at the edge of the cloud, many inside it
- **autoencoder** ($29 \to 16 \to 8 \to 16 \to 29$, trained on normal transactions): flagging the 1% largest errors catches 73 of 98 frauds (74%); 12.8% of the flagged transactions are fraud
- **ROC:** both scores have AUC ≈ 0.95; at 0.1% false alarms the autoencoder finds 44% of the frauds, the distance 18%

A bank can review only a small share of transactions, so what counts is the fraction of frauds found when very few normal ones are flagged.

Tumour Gene Expression Data

	gene_1	gene_2	gene_3	⋯	gene_20530	tumour type
	2.02	3.27	5.48	⋯	0.00	PRAD
$\mathbf{x}_i$	0.59	1.59	7.59	⋯	0.00	LUAD
	3.51	4.33	6.88	⋯	0.00	PRAD
	3.66	4.51	6.66	⋯	0.00	PRAD
	2.66	2.82	6.54	⋯	0.00	BRCA
			⋮			

$D = 20\,531$ genes

only to evaluate

One row per tumour sample: $\mathbf{x}_i$ = expression level of each gene (RNA sequencing), anonymised genes, values 0–20.8 (apparently a log scale; normalisation not documented). Tumour type: a label, used only to evaluate.

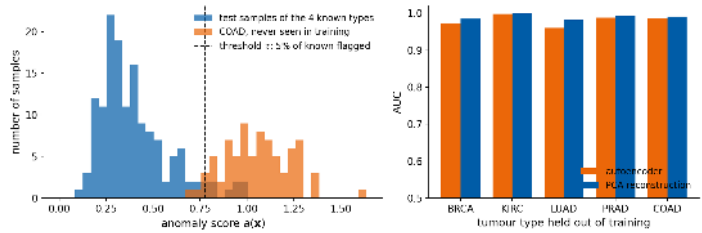
BRCA	breast invasive carcinoma
KIRC	kidney renal clear cell carcinoma
LUAD	lung adenocarcinoma
PRAD	prostate adenocarcinoma
COAD	colon adenocarcinoma

801 samples from The Cancer Genome Atlas (TCGA): BRCA 300, KIRC 146, LUAD 141, PRAD 136, COAD 78.

S. Fiorini, *gene expression cancer RNA-Seq*, UCI Machine Learning Repository (2016), doi:10.24432/C5R88H;

J. N. Weinstein et al., *The Cancer Genome Atlas Pan-Cancer analysis project*, Nat. Genet. **45**, 1113 (2013), doi:10.1038/ng.2764

Detection of an Unseen Tumour Type



- train on four types (80%); 2000 most variable genes, chosen and standardised on the training part only; autoencoder $2000 \to 256 \to 32 \to 256 \to 2000$
- COAD held out: τ flags 5% of the known test samples and 96% of the COAD samples; each type held out in turn: AUC 0.96–1.00, PCA reconstruction (32 components) 0.98–1.00
- the types differ in many genes: an unseen type lies far from the training data, an easy anomaly that PCA already finds

Methods without Labels

method	gives	new data	keeps / limits
PCA	linear projection, largest variance	projected with V_m	global structure; misses curved structure
t-SNE	a 2D map for looking	no: the map is refitted	neighbours; gaps and cluster sizes are not distances
k-means	k groups with centres	assigned to the nearest centre	round groups of similar size; k must be chosen
DBSCAN	groups of any shape, plus noise	not directly	dense regions; needs the radius ε and m
autoencoder	encoder, decoder, latent space	encoded and rebuilt	curved structure; reconstruction error flags anomalies

Common pipeline: PCA to a few tens of dimensions, then t-SNE or clustering; an autoencoder for curved structure or anomalies.

Summary

- **unsupervised learning** finds structure in data without labels: a few coordinates, groups, unusual samples
- **PCA**: a linear projection on the directions of largest variance; **t-SNE**: a non-linear map that keeps neighbours
- **k-means**: groups around k centres, k chosen by us (elbow, silhouette); it assumes round clusters; **DBSCAN** follows dense regions of any shape
- **autoencoder**: compress to a latent space and rebuild; the reconstruction error flags unusual samples, judged by the ROC curve and AUC
- **tables**: the same tools on card transactions and on tumour gene expression; an unseen tumour type is an anomaly

Next Lecture: Learning a Probability Distribution from Data

The data $\mathbf{x}_1, \ldots, \mathbf{x}_n$ are samples from an unknown distribution $p(\mathbf{x})$. Can we learn p itself, and draw *new* samples from it?

- probability, maximum likelihood and latent-variable models
- variational autoencoders: a latent space we can sample, new digits from random latent vectors
- conditional generation: a chosen digit or cell type
- disentangled latent features (β-VAE) as a tool for interpretation

The latent space of an autoencoder becomes a probability distribution we can draw new data from.

Thank You

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar



